Scanned Code Report

AUDITAGENT

Powered by NETHERMIND SECURITY

AUDITAGENT

Important Notice

This Report has been generated entirely by AI and has not been manually reviewed by Nethermind's security team. It does not constitute a full security audit. Projects may not represent or imply otherwise in any public communication. All findings, observations, and recommendations may contain errors or omissions and must be independently verified by a qualified human reviewer before being acted upon. This Report should be used as a supplementary tool only, alongside professional security practices.

Code Info

Auditor Scan

Scan ID 4	Date June 08, 2026
Organization CMTA	Repository SnapshotEngine
Branch dev	Commit Hash dec31b27...cdcaeb7d

Contracts in scope

contracts/base/CMTATInternalSnapshotBase.sol
contracts/base/SnapshotEngineBase.sol
contracts/deployment/CMTATStandaloneInternalSnapshot.sol
contracts/deployment/CMTATUpgradeableInternalSnapshot.sol
contracts/deployment/SnapshotEngine.sol
contracts/deployment/SnapshotEngineOwnable2Step.sol
contracts/library/Errors.sol
contracts/library/SnapshotBase.sol
contracts/library/SnapshotStateInternal.sol
contracts/interface/IERC20SnapshotCompatible.sol
contracts/interface/ISnapshotBase.sol
contracts/interface/ISnapshotScheduler.sol
contracts/interface/ISnapshotState.sol
contracts/modules/SnapshotSchedulerModule.sol
contracts/modules/SnapshotStateModule.sol
contracts/modules/SnapshotUpdateModule.sol
contracts/modules/VersionModule.sol

Code Statistics

Findings 1	Contracts Scanned 17	Lines of Code 1350

Findings Summary

Total Findings

- High Risk (0)
- Medium Risk (0)
- Low Risk (0)
- Info (1)
- Best Practices (0)

Code Summary

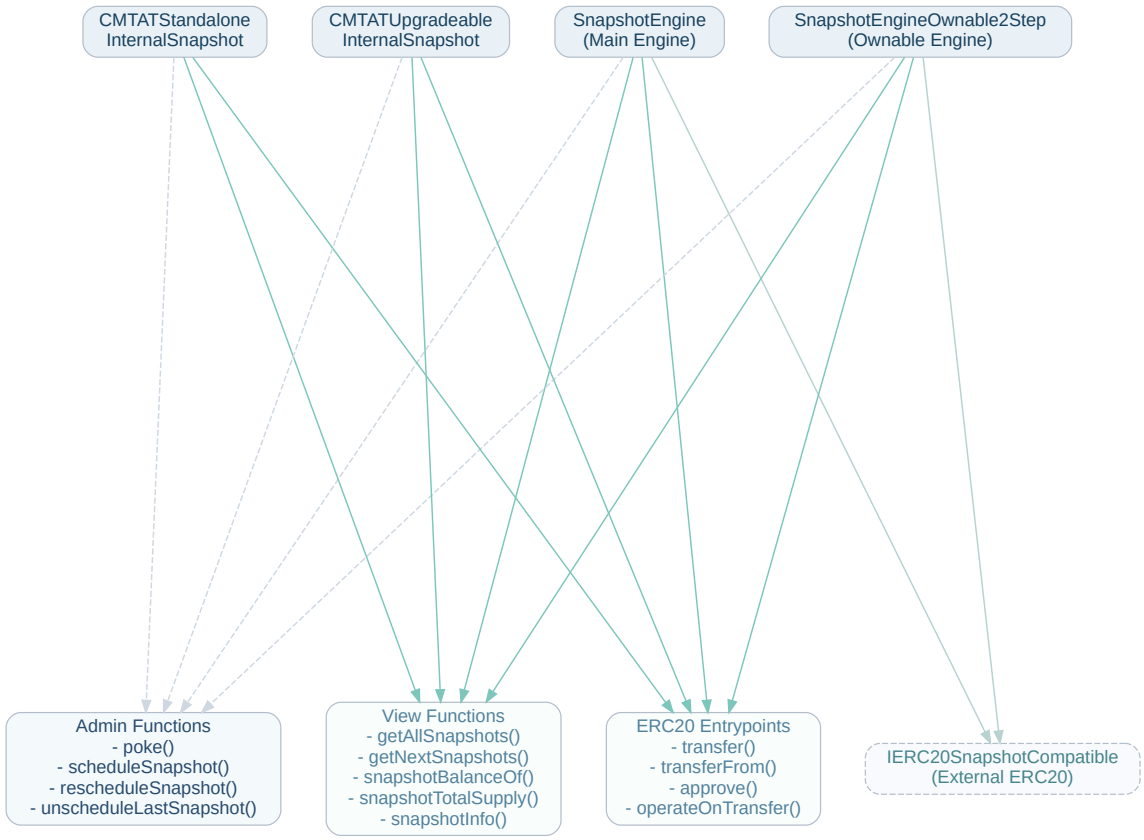
The protocol implements a snapshotting engine designed for ERC-20 tokens (specifically integrating with the CMTAT standard). It allows for scheduling, modifying, and canceling snapshots of token balances and total supply at precise historical timestamps instead of block numbers or discrete IDs.

The system consists of modules to handle snapshot state queries, scheduling operations, and balance updates on token transfers, mints, and burns. It can be deployed internally as part of a standalone token contract (e.g., `CMTATStandaloneInternalSnapshot` or `CMTATUpgradeableInternalSnapshot`) or externally bound to a compatible ERC-20 token using deployment wrappers like `SnapshotEngine` or `SnapshotEngineOwnable2Step`.

Main Entry Points and Actors

- `transfer(to, value)`: Called by Token Holders to transfer tokens.
- `transferFrom(from, to, value)`: Called by Spenders to transfer tokens on behalf of a Token Holder.
- `approve(spender, value)`: Called by Token Holders to approve a spender.
- `poke()`: Called by Snapshot Managers (SNAPSHOOTER_ROLE) to manually materialize the latest eligible scheduled snapshot.
- `scheduleSnapshot(time)`: Called by Snapshot Managers to schedule a snapshot at a specific time.
- `scheduleSnapshotNotOptimized(time)`: Called by Snapshot Managers to schedule a snapshot without optimization assumptions.
- `rescheduleSnapshot(oldTime, newTime)`: Called by Snapshot Managers to change the time of a scheduled snapshot.
- `unscheduleLastSnapshot(time)`: Called by Snapshot Managers to cancel the last scheduled snapshot.
- `unscheduleSnapshotNotOptimized(time)`: Called by Snapshot Managers to cancel a scheduled snapshot without optimization assumptions.
- `operateOnTransfer(from, to, balanceFrom, balanceTo, totalSupply)`: Called by the bound ERC-20 token contract to update snapshot states during token transfers (in external engine deployments).

Code Diagram



1 of 1 Findings

contracts/library/SnapshotBase.sol contracts/modules/SnapshotUpdateModule.sol contracts/base/SnapshotEngineBase.sol contracts/base/CMTATInternalSnapshotBase.sol

Unbounded scan of overdue scheduled snapshots can make transfer hooks and poke run out of gas

Info

The transfer hook always attempts to advance the snapshot pointer before updating account snapshots. `SnapshotUpdateModule._snapshotUpdate()` calls `SnapshotBase._setCurrentSnapshot()`, and `_setCurrentSnapshot()` calls `_findScheduledMostRecentPastSnapshot()`. That helper linearly scans `_scheduledSnapshots` from `_currentSnapshotIndex` until it reaches the first future timestamp:

`function _snapshotUpdate(...) internal virtual {
SnapshotBase._setCurrentSnapshot();
...
}

for (uint256 i = $._currentSnapshotIndex; i < currentArraySize; ++i) {
if ($._scheduledSnapshots[i] <= block.timestamp) {
mostRecent = $._scheduledSnapshots[i];
index = i;
} else {
break;
}
}`

If a large backlog of scheduled snapshots becomes overdue while no transfer or `poke()` advances the pointer, every later transfer, mint, burn, or `poke()` must scan the entire overdue suffix in one transaction. When the backlog is large enough to exceed the block gas limit, the call fails before `_currentSnapshotIndex` is updated, so subsequent attempts start from the same stale index and fail again. The due snapshots also cannot be removed through the unscheduling functions because `_checkTimeSnapshotAlreadyDone(time)` reverts once `time <= block.timestamp`. In the internal CMTAT variant this blocks `_update()` before the ERC-20 balance update, and in the external-engine variant it makes the bound token's `operateOnTransfer()` call fail and roll back the token transfer.

Disclaimer

No Guarantee of Accuracy or Completeness. Kindly note, no guarantee is being given as to the accuracy and/or completeness of any of the outputs the AuditAgent may generate, including without limitation this Report. The results set out in this Report may not be complete nor inclusive of all vulnerabilities. The AuditAgent is provided on an 'as is' basis, without warranties or conditions of any kind, either express or implied, including without limitation as to the outputs of the code scan and the security of any smart contract verified using the AuditAgent.

This Report is not a security audit. This Report has been generated automatically by AuditAgent, an AI-powered automated code scanning tool. It does not constitute, and must not be represented or relied upon as constituting, a full or comprehensive security audit conducted by Nethermind or any of its personnel. A formal security audit involves manual review by experienced human auditors and is a materially different service. If you require a full security audit, please contact Nethermind's security audit team at auditagent@nethermind.io

Use of Nethermind's name. "Nethermind" is a trade mark of Demerzel Solutions Limited. Use of this Report does not authorise you to represent that your code or smart contract has been "audited by Nethermind" or to use any similar phrase. Any such representation would be false, misleading, and an infringement of Nethermind's intellectual property rights.]
No Endorsement or Security Guarantee. Blockchain technology remains under development and is subject to unknown risks and flaws. This Report does not indicate the endorsement of any particular project or team, nor guarantee its security. Neither you nor any third party should rely on this Report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset.

Disclaimer. To the fullest extent permitted by law, Nethermind disclaims any liability in connection with this Report, its content, and any related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. Nethermind does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and Nethermind will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate.

FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.